一條時間線，四個不同的問題

CPU 排程先決定「下一個誰能執行」，再討論公平、效率或期限。**同一張時間表，可以讓平均等待變短，也可能讓某個工作更晚完成。** 本章會逐次列出選擇依據，再計算結果。

學習順序	要能自己回答的問題
狀態與事件	工作還在等 CPU，還是在等 I/O？到達是否一定搶占？
一般排程	FCFS、SJF、SRTF、RR 與回饋佇列，當下各選誰？
多核心與即時	遷移成本、工作相依與期限，如何改變判斷？
系統案例	版本、計時器與權重，如何產生具體時間線？

前置知識是行程／執行緒狀態、上下文切換，以及基本代數。英文術語在第一次出現時說明；數值例子均列單位與成立條件。

6.1　先分清 ready、running 與 blocked

Ready（就緒）表示資源條件已滿足，只差 CPU；running（執行中）表示正在占用 CPU；blocked（阻塞）表示還在等 I/O 或某個事件，此時多給 CPU 也無法繼續。

一次 CPU burst 是工作從可執行狀態算到下次阻塞或完成所需的一段 CPU 需求，搶占只是暫停這段需求，並不把它完成或重算。短期排程器從可執行者中選下一個；dispatcher（分派器）再保存／恢復上下文並移交控制權。

Ready queue 是「可選集合」的概念名稱，實作不一定是 FIFO：也可能是優先佇列或樹。單 CPU 任一時刻至多有一個 running 工作；同一工作不能同時出現在 ready 與 running。

哪些事件必須換人？哪些只是重新比較？

狀態改變與選取決策是兩個步驟



新到達 → Ready；完成 → 離開系統（圖中省略出口）

Blocked 不在 ready queue；搶占者保留剩餘 CPU 需求。

Running 工作阻塞或完成後，CPU 才能交給另一個 ready 工作；若沒有候選，就閒置。新工作到達或 I/O 完成則只增加候選，**是否搶占，要再看政策與比較鍵**。

非搶占政策讓目前工作執行到阻塞、完成或主動讓出；可搶占政策允許提早收回 CPU。「blocked→ready 發生了」本身不等於已把目前工作趕下 CPU。

切換成本：忙碌，不一定在推進應用程式

單 CPU，P1 先執行 5 ms 後阻塞，P2 需要 2 ms，兩者切換需 1 ms：

區間（ms）	CPU 在做什麼？	應用工作是否前進？
[0,5)	P1	是，5 ms
[5,6)	保存／恢復上下文	否，1 ms 成本
[6,8)	P2	是，2 ms

整段沒有閒置，忙碌率為 8/8=100%；但有用工作占比只有 7/8=87.5%。兩個比例的分子不同，不能都稱為「效率」。本例不把 P1 的後續 I/O 完成當成已知，也不宣稱它已完成整個工作。

長期 admission（接納）決定哪些工作進系統；短期排程決定下一個執行者；傳統中期排程可暫停／換出工作以降低記憶體壓力。三者影響的時間尺度不同。

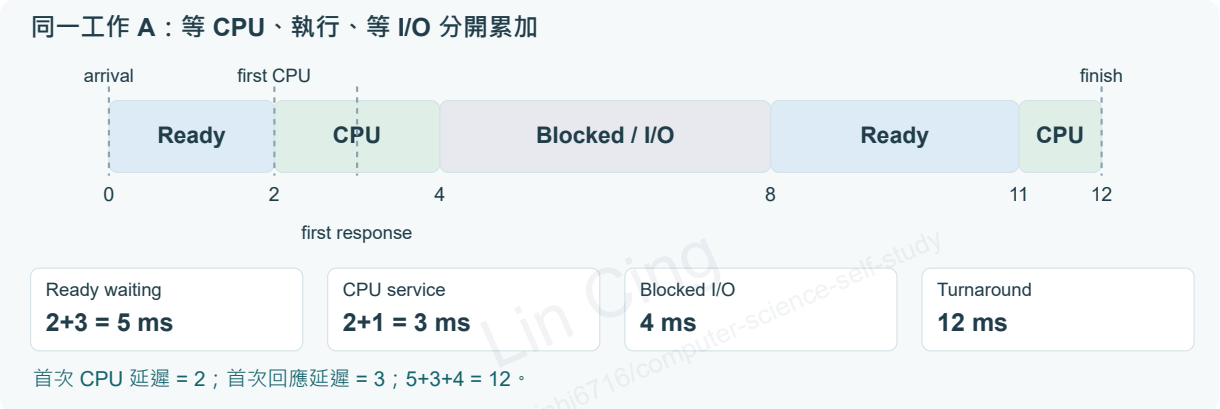
6.2　先固定指標，再比較政策

令 a 為到達、s 為首次取得 CPU、o 為首次產生回應、f 為完成時刻。

指標	計算方式	不應包含或混用的量
等待時間 W	所有 ready 區間的總長	不含 CPU 執行及 blocked 時間
周轉時間 T	f−a	包含途中等待與 I/O
首次 CPU 延遲 R_CPU	s−a	不能推定應用程式已輸出
首次回應延遲 R_out	o−a	不等於全部輸出或工作完成

本章數值排程題若只寫 response，固定採 R_CPU。 原書 §6.2 從互動請求的首次回應說明 response；它與第一次分派 CPU 不必同時。題目若量輸出，會明寫 R_out，不把教材的計算慣例當作兩者恆等。

等 I/O 的四毫秒，為什麼不算 ready waiting？



工作 A 在 0 到達，ready [0,2)、CPU [2,4)、blocked [4,8)、ready [8,11)、CPU [11,12)，首次回應在 3。故 W=2+3=5、CPU service=3、blocked=4、T=12，且 **5+3+4=12**。

R_CPU=2，R_out=3。首次取得 CPU 並未立即回應；首次回應時也尚未完成。若把 I/O 的 4 ms 算進等待，會誤得 W=9；只有沒有 blocked 等其他區間時，才可直接用 T−CPU service 求 W。

6.3　排程不是背名稱，而是固定選取規則

以下一般排程例子固定單 CPU、每工作一段 CPU burst、已知需求、忽略切換成本。除特別指定外，同鍵按到達順序，再按編號；區間一律寫成 [開始,結束)。

政策	比較什麼？	何時重新選取？
FCFS	ready 入列順序	目前工作讓出 CPU 時
非搶占 SJF	下一段 CPU burst 長度	目前工作讓出 CPU 時
SRTF	尚需的 CPU 時間	到達、完成等事件；更短者可搶占
RR	FIFO 次序與剩餘時間片	完成、阻塞或時間片用完
Priority	明定方向的優先權	依政策決定是否搶占

把 burst 視為已知是教學假設。實機通常只能估計，錯估時不能沿用精確長度下的最適保證。

FCFS：長工作先進來，短工作也得等

P1/P2/P3 都在 0 抵達，CPU 需求為 8/4/2 ms，FCFS 入列順序為 P1、P2、P3。

工作	執行區間	完成 f	等待 W	周轉 T
P1	[0,8)	8	0	8
P2	[8,12)	12	8	12
P3	[12,14)	14	12	14

平均等待 20/3≈6.67 ms，平均周轉 34/3≈11.33 ms。後方短工作一起等長工作，是 convoy effect 的基本現象；若再交錯 I/O，還可能降低裝置與 CPU 的配合效率。

不能因此稱 FCFS 在所有工作負載下「最差」。在每筆前方只有有限工作、每段需求有限的 FIFO 模型中，可證有限等待；沒有到達或服務上界，就沒有共同的數值等待上限。

SJF：同一批輸入，先做短者

沿用 8/4/2 ms，但順序改為 P3→P2→P1。下表仍以 **P1、P2、P3** 排列，避免把執行順序誤當工作編號。

工作	執行區間	完成 f	等待 W	周轉 T
P1	[6,14)	14	6	14
P2	[2,6)	6	2	6
P3	[0,2)	2	0	2

平均等待 8/3≈2.67 ms，比 FCFS 少 4 ms；但 P1 的完成時刻從 8 變成 14。平均值改善，不代表每一個人都改善。

SJF 的交換論證，在哪些條件下成立？

所有工作同時可用、單一 CPU burst、已知長度、非搶占且沒有切換成本。考慮相鄰兩工作 x>y，它們開始前已有共同等待 h。

順序	第一筆等待	第二筆等待	兩筆等待總和
x 再 y	h	h+x	2h+x
y 再 x	h	h+y	2h+y

交換後少 x−y>0；兩筆總執行長度不變，所以後方工作的開始時間不變。反覆消除逆序，得到短者在前的排列，因此等待總和最小。工作數固定，平均等待也最小；總 CPU 需求固定，平均周轉隨之最小。

不同到達、允許刻意閒置、權重、期限或估計誤差，都不能未經推導直接套用此結論。

不知道下一段 burst，怎麼估計？

以指數平均保存歷史：$\hat{b}_{n+1} = \alpha b_n + (1-\alpha)\hat{b}_n$，其中 b_n 是剛量到的 burst、$\hat{b}_n$ 是先前估計，$0 \le \alpha \le 1$。

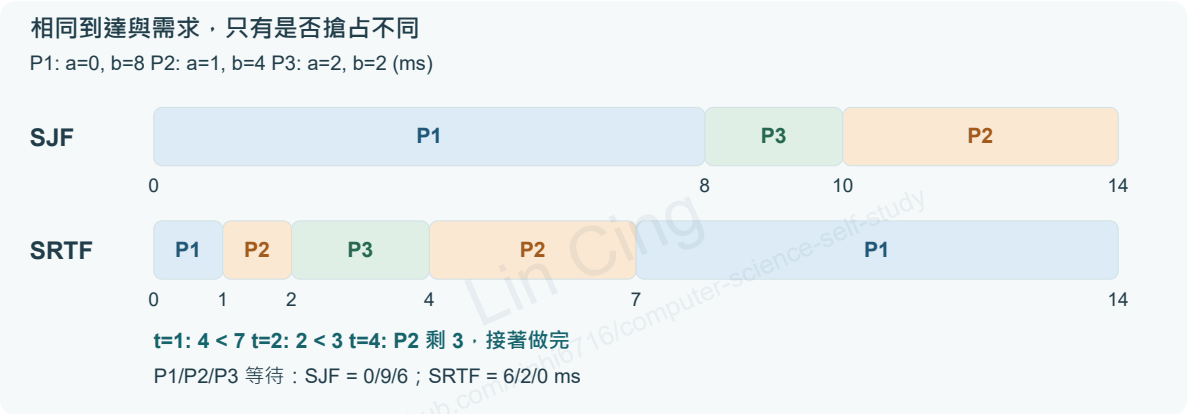
α=1/2，初值估計 10 ms	本次量到 b_n	下一次估計
第一次	6	(6+10)/2=8
第二次	4	(4+8)/2=6
第三次	8	(8+6)/2=7

展開遞迴後，舊觀測的係數為 $\alpha(1-\alpha)^j$；$0<\alpha<1$ 時愈舊權重愈小。$\alpha=0$ 完全忽略新觀測，$\alpha=1$ 只用最近一次。

預測不是保證：工作突然進入較長計算階段時，舊歷史可能低估需求。排程器須另外處理公平與飢餓，不能把「估計最短」當成「實際最短」。

SRTF：新來者要和「剩餘量」比較

固定 P1=(到達0,需求8)、P2=(1,4)、P3=(2,2)，單位 ms。同剩餘量時保留 current；否則選最短者。



t=1，P1 剩 7，P2 需 4，所以搶占；t=2，P2 剩 3，P3 需 2，再搶占。t=4 P3 完成，剩餘量是 P2=3、P1=7，先恢復 P2；**不是把 P2 從原始 4 重新開始**。

SRTF：逐事件重建時間表與指標

事件時刻	可選者的剩餘量	下一段	理由
0	P1=8	P1 [0,1)	唯一候選
1	P1=7、P2=4	P2 [1,2)	4<7
2	P1=7、P2=3、P3=2	P3 [2,4)	2 最小
4	P1=7、P2=3	P2 [4,7)	3<7
7	P1=7	P1 [7,14)	唯一候選

依 P1/P2/P3，完成為 14/7/4、周轉 14/6/2、等待 6/2/0，平均等待 8/3 ms；首次 CPU 延遲皆為 0。非搶占 SJF 則完成 8/14/10，等待 0/9/6，平均 5 ms。兩者的到達與需求完全相同，差別才可歸因於政策。

RR：未完成者才回隊尾

P1/P2/P3 皆在0到達，需求5/2/3 ms，初序P1→P2→P3、q=2、成本0。完成者離開，其他工作用滿片額才回隊尾。



依 P1/P2/P3，RR 完成10/4/9、等待5/2/6、首次 CPU 延遲0/2/4。t=4 的P2已完成，不再入列。SJF平均等待7/3，RR為13/3 ms；兩者工作量皆10 ms。

RR：回應上界與切換成本要分開算

固定 n 筆始終就緒的工作，時間片 q、每次交接成本 c。某工作剛回到隊尾，在再次開始前至多有 n−1 個其他片段；若把到它的交接也計入，等待上界為 **(n−1)q+nc**。這是固定候選集、無更高類別插入且無阻塞的模型上界。

持續有其他工作，c=0.1 ms	q	一個滿片額的有用時間占比 q/(q+c)
較大片額	4 ms	4/4.1≈97.56%
較小片額	0.1 ms	0.1/0.2=50%

q 大到每筆 burst 都能一口氣完成時，RR 對這個單 burst 工作集退化成 FCFS。q 小能縮短輪候，卻增加交接占比；只剩一個 runnable 工作時，時間片到期不一定有跨工作切換，不能照表硬加成本。

Priority、aging 與「等很久」的差別

優先權數字方向必須先約定；有的系統數字大者優先，有的相反。固定優先權若持續有高優先權工作 ready，低者可能永遠輪不到。

有限追蹤只能證明「到目前仍未執行」，不能證明無限飢餓。反例模型：低者 L 在 0 就緒，高者 H_k 在每個整數 k 到達、各需 1 ms；若高者可完全占滿 CPU，L 永不前進。

Aging（等待愈久逐步提高優先權）必須配合有上限的等級、同級公平、有限前方工作與服務需求等條件，才可推出有限等待；只寫「會提高」不構成上界。即時系統若改動固定優先權，還須重做期限分析。

多級佇列與回饋佇列：差在工作能不能換級

Multilevel queue 把類別固定分在不同佇列；各級內部及級間都需要選取規則。Multilevel feedback queue（MLFQ，多級回饋佇列）則依執行／等待行為換級。

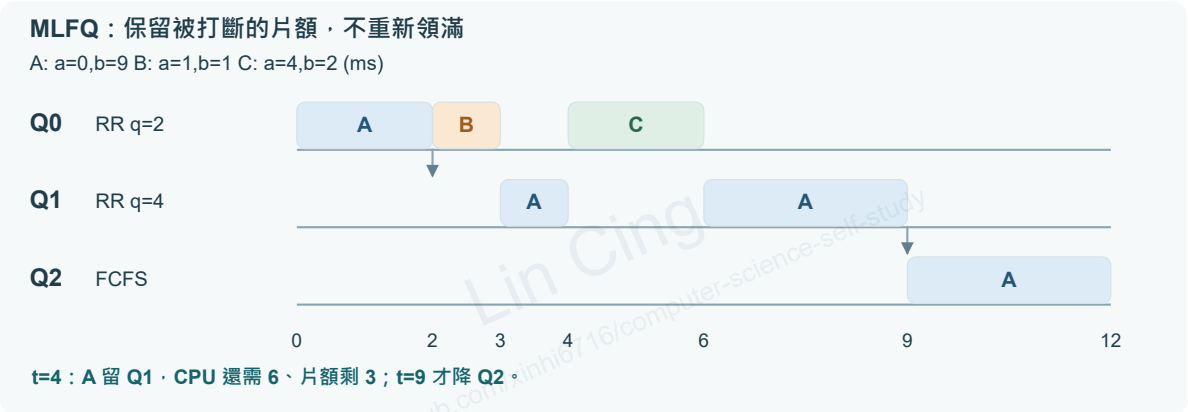
以下採一個明確的教學變體，**不是宣稱所有 MLFQ 都相同**：

規則	本例定義
由高到低	Q0：RR q=2；Q1：RR q=4；Q2：FCFS
新到達	進 Q0；高級非空就搶占低級
同級片額用完	未完成者降下一級；Q2 留原級
被高級搶占	留在原級隊首，保留剩餘片額與需求
升級／阻塞	本例沒有阻塞；有限追蹤內不升級

不同的重新入列、升級與片額重設規則，都可能改變答案。

MLFQ：被搶占不等於片額重新領滿

令 A=(到達0,需求9)、B=(1,1)、C=(4,2)，單位 ms，沿用前頁規則。



A 在 Q0 做 [0,2)，剩 7 並降 Q1；B 在同級不插隊，於 [2,3) 完成。A 在 Q1 做 [3,4) 後，被新進 Q0 的 C 搶占，**剩餘片額是 3，需求是 6，兩者不同**。

C 在 [4,6) 完成；A 回 Q1 做 [6,9)，用完餘下 3 ms，仍需 3，才降 Q2 並在 [9,12) 完成。

MLFQ：每次移動都能核對佇列與預算

表中列事件處理後、分派前的佇列；括號為「剩餘 CPU／該級剩餘片額」，— 表示 Q2 不設片額。

t	Q0	Q1	Q2	下一段
0	A(9/2)	空	空	A 0–2
2	B(1/2)	A(7/4)	空	B 2–3
3	空	A(7/4)	空	A 3–4
4	C(2/2)	A(6/3)	空	C 4–6
6	空	A(6/3)	空	A 6–9
9	空	空	A(3/—)	A 9–12

若每次被高級打斷都重領 Q1 的 4 ms，A 會延後降級，已是另一種政策。沒有 aging／boost 的嚴格級間優先權，也不能保證低級在無限到達下不飢餓。

6.4　誰在排程 user thread？誰在排程 CPU？

User-level thread 由函式庫管理；核心選取的是可被核心辨識的執行實體。Process-contention scope（PCS）在行程內分配可用執行實體；system-contention scope（SCS）在系統層競爭 CPU。

映射	T1 阻塞時，T2 能否繼續？	多 CPU 能否同時執行兩者？
Many-to-one	若阻塞唯一核心執行實體，就不能	只有一個核心實體，不能
One-to-one	T2 自己的核心執行實體仍 ready 才能	還要有至少兩個可用 CPU

例如 T2 需 4 ms，T1 等網路；one-to-one 也不能無條件說 T2 於 4 完成，還須假設它從 0 起獨占一個 CPU。映射提供執行的可能性，政策與負載才決定實際時間。

6.5　多核心：多給 CPU，還要付資料搬動成本

Processor affinity（處理器親和性）讓工作留在熟悉的快取／TLB 上；load balancing（負載平衡）則把工作搬到較空的 CPU。兩者有取捨，不能只看佇列長度。

同一工作不能被兩個 CPU 同時取走。遷移必須在同步保護下移除舊入列、移交唯一擁有者，再加入目的佇列；已在執行的工作與尚未開始者也不能混作同一種「偷取」。

以下保留一個可完整重算的八核心例子：所有工作在 0 就緒，不可切分，單工作不得同時跑在兩核；每次移交與快取暖機合計 1 ms。這些是假定成本，並非量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

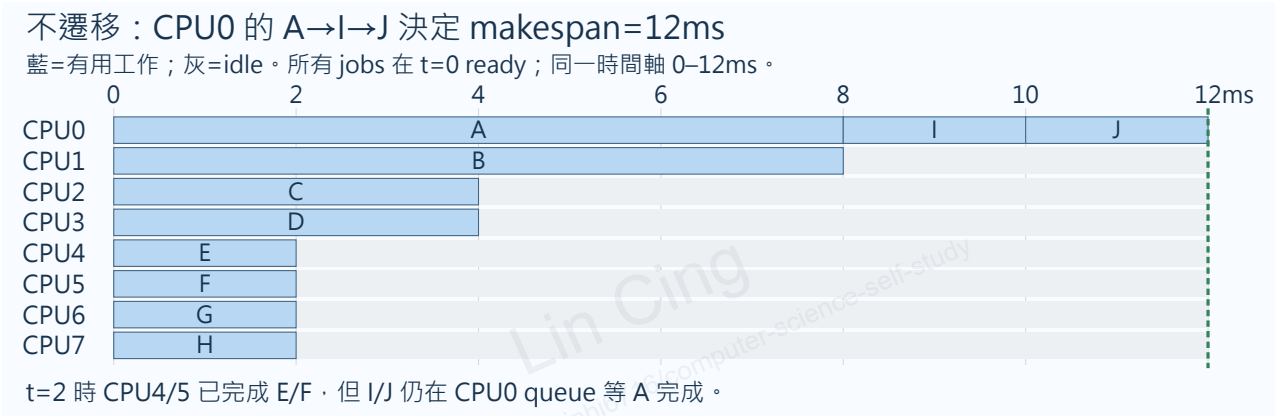
八核心：移交待執行工作，能省下多少時間？

固定8cores、所有jobs在t=0就緒、不可切分、單job不得同時跑在兩core。Jobs A/B各8ms、C/D各4ms、E/F/G/H/I/J各2ms，共36ms有用工作。初始CPU0 queue=[A,I,J]；CPU1至7分別只有B、C、D、E、F、G、H。

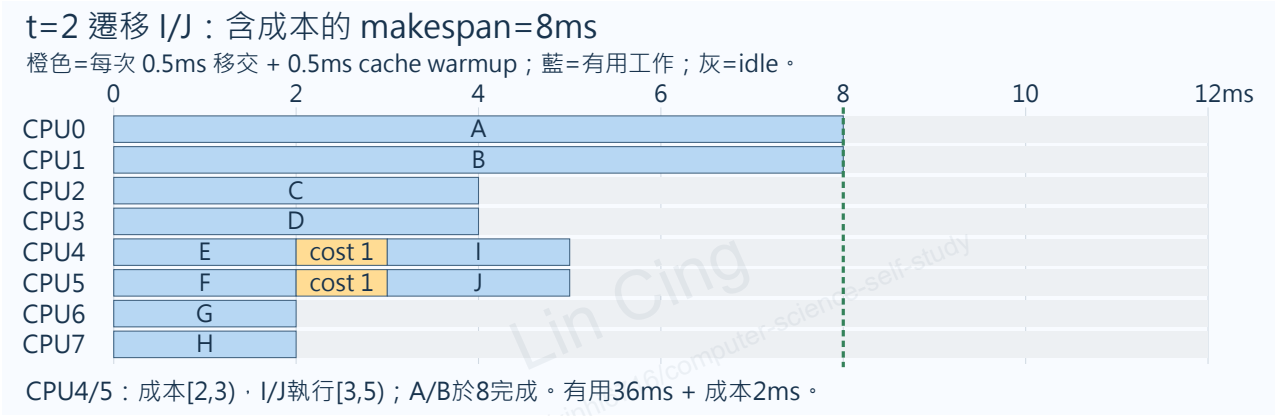
不遷移時CPU0依序A[0,8)、I[8,10)、J[10,12)，makespan=12ms。t=2時CPU4/5已做完E/F；在queue lock下各移除尚未執行的I/J，移交其唯一owner。每次遷移明定0.5ms queue/transfer加0.5ms cache warmup，故兩core各在[2,3)付成本、[3,5)執行I/J。CPU0只執行A至8，makespan降為8ms，speedup=12/8=1.5；有用工作仍36ms，新增成本共2ms。這是給定成本的教學模型，不是實機測量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搬移前：誰在等待？



搬移後：連同成本重新計時



單job A長8ms給下界T≥8，所以本例達到此下界。若每次遷移成本改為c，makespan=max(8,2+c+2)=max(8,4+c)；c<8才比12ms嚴格改善。資料相依、硬性affinity或不同cache成本會改變模型，不能只因core閒置就宣稱遷移一定有益。

6.6　即時排程：先分清 task、job 與期限

週期工作（task）τ_i 反覆產生一筆筆待執行的 job。以下統一使用執行時間上界 C_i、週期 T_i、相對期限 D_i。從時刻 0 同步釋放時，第 k 筆 job（k 從 0 起）的釋放時刻與絕對期限為

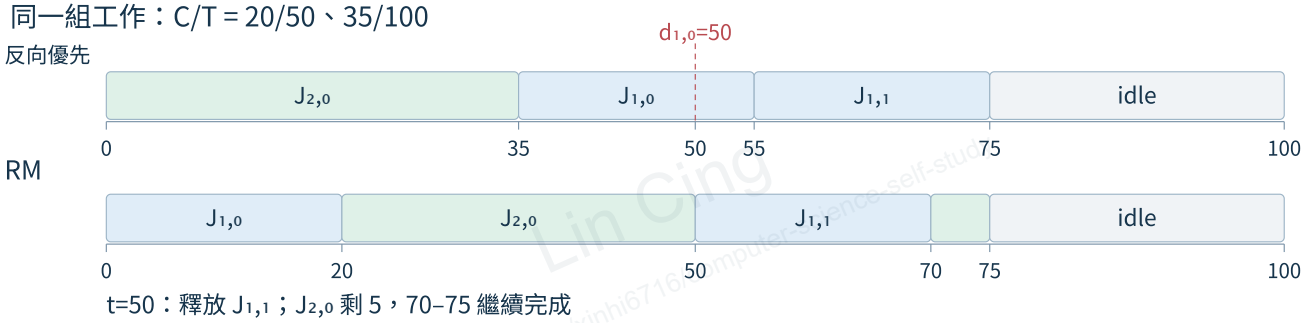
$$r_{i,k} = kT_i, \qquad d_{i,k} = r_{i,k} + D_i.$$

本節基本模型：單 CPU、可立即搶占、工作彼此獨立，沒有鎖阻塞、I/O、自我暫停、切換或中斷成本；除末頁外 $D_i=T_i$。 到達時只把新 job 放進 ready set，未必換人執行。完成恰在期限上算成功；時刻 t 先計入 [前一事件,t) 的執行，再處理 t 的到達。

量	問的問題	不能混用的量
C_i	每筆最多需要多少 CPU 時間？	不是 response time
T_i	同一 task 隔多久產生下一筆？	不是當前 job 剩餘期限
D_i	從釋放起多久必須完成？	絕對期限為 $r_{i,k}+D_i$
$U=\Sigma C_i/T_i$	長期 CPU 需求占比是多少？	不單獨描述短時間擁塞

RM：週期短的 task 優先，不是剩餘工作少者優先

Rate-monotonic（RM）把**較短週期設為較高固定優先權**；新來的高優先權 job 可搶占。令 τ_1=(C_1=20,T_1=D_1=50)、τ_2=(35,100,100)，時間單位均為 ms。



若把 τ_2 排在前面，$J_{1,0}$ 只能在 [35,55) 執行，超過期限 50。RM 則先做 $J_{1,0}$；$J_{2,0}$ 在 [20,50) 已做 30，t=50 被 $J_{1,1}$ 搶占，**剩下的 5 不會重做**，於 [70,75) 完成。

RM 的完成時刻為 $J_{1,0}$=20、$J_{1,1}$=70、$J_{2,0}$=75，對應期限 50、100、100。t=100 是共同週期邊界，舊 job 全已完成；同一 ready 狀態隨後重現，故整個排程可每 100 ms 重複。

RM 利用率界：通過可保證，沒通過要再分析

在本節獨立、可搶占、$D_i=T_i$ 的理想單 CPU 模型，n 個週期 task 的經典**充分條件**為

$$U = \sum_i C_i/T_i \le n(2^{1/n} - 1).$$

n=2 時界為 $2(\sqrt{2}-1)\approx 0.8284$；前例 U=20/50+35/100=0.75，通過。$n\to\infty$ 時界趨近 $\ln 2\approx 0.6931$。這些數字是對所有符合模型工作集的保證界，**不是每個 RM 工作集最多只能用這麼多 CPU**。

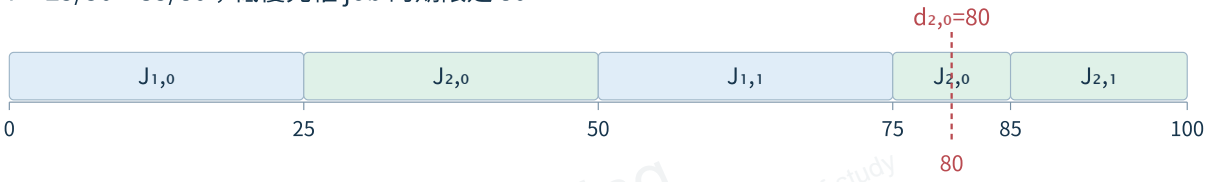
反例：τ_1=(C=1,T=D=2)、τ_2=(2,4,4)，U=1。RM 在 [0,1) 做 $J_{1,0}$，[1,2) 做 $J_{2,0}$，[2,3) 做 $J_{1,1}$，[3,4) 做 $J_{2,0}$ 的剩餘 1。三筆完成於 1、3、4，期限分別為 2、4、4。t=4 無積欠且同時再釋放，故可重複，RM 仍可排程。

判斷順序：先核對假設，再用充分測試；不通過時，以更精確的干擾分析或可證明重複的時間表判定。單憑 U>0.8284 不能宣告失敗；U≤1 也不能保證任意固定優先權成功。

同樣不到滿載，RM 仍會讓某筆 job 超時

改成 τ_1=(25,50,50)、τ_2=(35,80,80)。U=25/50+35/80=15/16=0.9375；RM 仍讓 τ_1 優先。

RM：C/T = 25/50、35/80；低優先權 job 的期限是 80



t=80：舊 $J_{2,0}$ 還剩 5；新 $J_{2,1}$ 另行加入

$J_{2,0}$ 在 [25,50) 做了 25，剩 10；[50,75) 被 $J_{1,1}$ 搶占；[75,80) 只能再做 5，**到了期限 80 仍剩 5**，所以此時已失敗。若超時 job 不丟棄，續做到 85 才完成。t=80 釋放的 $J_{2,1}$ 與舊 $J_{2,0}$ 是兩筆不同 job，圖中沒有把它們合併。

U<1 只表示長期總量未超過 CPU 供給；固定優先權仍可能把某個短區間擠滿，讓低優先權 job 錯過期限。證明失敗的是 t=80 的剩餘需求，不是「超過 RM 充分界」。

用干擾次數核對 RM 的完成時刻

只分析兩個同步釋放的 task，且 τ_1 優先、$C_1 \le T_1$、沒有阻塞。$J_{2,0}$ 在完成前長度 R 的區間 [0,R) 內，τ_1 於 $0, T_1, 2T_1, \cdots$ 釋放；共有 $\lceil R/T_1 \rceil$ 筆會干擾。**在 R 才釋放的新 job 不算入先前區間**。

$$w^{(0)} = C_2, \qquad w^{(k+1)} = C_2 + \left\lceil w^{(k)}/T_1 \right\rceil C_1.$$

從 C_2 往上代入，求出這筆同步 job 的最早固定點。若遞增值已超過 D_2，期限內不可能完成；固定點 $\le D_2$ 才通過此筆檢查。

迭代不減：右側對 w 單調，且首步不小於 C_2。若存在期限內固定點 R*，歸納得每一步 $w \le R^*$；因此迭代超出期限即可排除期限內固定點。這裡只推導同步兩 task 的首筆；任意相位、多 task 或有 blocking 的完整最壞回應時間分析，還需要相應的 critical-instant 與干擾模型，不能由這張表直接推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三組工作集，逐次代入核對

工作集	$w^{(0)}$	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	核對結果
(20,50)、(35,100)	35	55	75	75	75≤100，對上圖
(25,50)、(35,80)	35	60	85	85	85>80，已超時
(1,2)、(2,4)	2	3	4	4	4=4，恰在期限完成

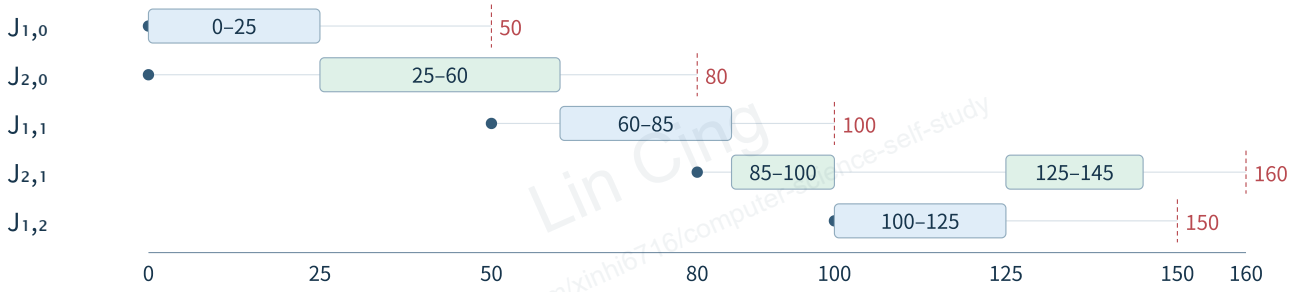
例如第二列：35+⌈35/50⌉25=60，下一步35+⌈60/50⌉25=85；85>80，已超過期限。第一列則在75固定，且75≤100。數值迭代的意義來自前頁的干擾上界，不能只背表格結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EDF：比較的是每筆 job 的絕對期限

Earliest-deadline-first（EDF）每次從 ready jobs 選**絕對期限最早**者。沿用 τ_1=(25,50,50)、τ_2=(35,80,80)；平手時先維持 current，否則依 task 編號、job 編號。

EDF：● 釋放；虛線為各 job 的期限；色塊才是實際執行



t=50 新 $J_{1,1}$ 的期限 100 比 current $J_{2,0}$ 的 80 晚，故 **$J_{2,0}$ 續作至 60 完成**。t=100 新 $J_{1,2}$ 的期限 150 比 current $J_{2,1}$ 的 160 早，才搶占。

圖只展開前 150 ms；有限前綴通過不代表未來全通過，後面的 EDF 定理才給一般保證。被搶占者保留剩餘量，不重新從 C_i 開始。

EDF 每一次選擇，都能回到 ready set 核對

表中列事件處理後、下一段執行前的 ready jobs；括號依序為「剩餘 CPU，絕對期限」。只列尚未完成的 job。

t (ms)	ready set	下一段執行	選擇理由
0	$J_{1,0}$(25,50)、$J_{2,0}$(35,80)	$J_{1,0}$：0–25	50<80
25	$J_{2,0}$(35,80)	$J_{2,0}$：25–50	唯一 ready
50	$J_{1,1}$(25,100)、$J_{2,0}$(10,80)	$J_{2,0}$：50–60	80<100，續作
60	$J_{1,1}$(25,100)	$J_{1,1}$：60–80	唯一 ready
80	$J_{1,1}$(5,100)、$J_{2,1}$(35,160)	$J_{1,1}$：80–85	100<160，續作

前五個事件的重點：$J_{2,0}$ 在 50 面對新到達仍續作；$J_{1,1}$ 在 80 也續作。到達只改 ready set，只有 current 不再是最早到期者時才搶占。

EDF 在 100 搶占：同一規則為何得到不同選擇？

仍以（剩餘 CPU，絕對期限）列 ready jobs。

t (ms)	ready set	下一段執行	選擇理由
85	$J_{2,1}$(35,160)	$J_{2,1}$：85–100	唯一 ready
100	$J_{1,2}$(25,150)、$J_{2,1}$(20,160)	$J_{1,2}$：100–125	150<160，搶占
125	$J_{2,1}$(20,160)	$J_{2,1}$：125–145	唯一 ready
145	空	idle：145–150	沒有可執行 job

J_1 的前三筆完成於 25、85、125，期限 50、100、150；J_2 的前兩筆完成於 60、145，期限 80、160。**job 到達、優先權重排、實際切換是三件事**；50 與 80 都有到達，但 current 仍是最佳選擇。

為什麼理想 EDF 在 D=T 時只需 U≤1？

考慮前述單 CPU 模型，自 t=0 無積欠開始，允許不同 task 有不同非負釋放相位。若 U>1，長度 t 內到期的需求為 Ut−O(1)，t 夠大就超過 t，故任何排程都不可能永遠準時。

反過來，假設 U≤1，EDF 卻首次在時刻 b 錯過期限。只看絕對期限 ≤b 的 jobs；向前找它們**持續有未完成工作**的最後一段 [a,b)，長度 L=b−a。在 a 前沒有這類積欠，故區間內要完成的這類 jobs 都在 a 或之後釋放。EDF 在整段只執行這類工作。

因 $D_i=T_i$，同一 task 在 [a,b] 內「釋放且到期」的 jobs 最多 $\lfloor L/T_i \rfloor$ 筆：首筆最快在 $a+T_i$ 到期，以後每次至少再隔 T_i。因此必須在這段處理的總需求滿足

$$\sum_i N_i C_i \le \sum_i \lfloor L/T_i \rfloor C_i \le L \sum_i C_i/T_i = LU \le L.$$

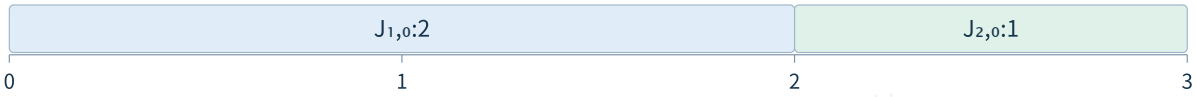
但首次超時表示 EDF 已連做 L 時間仍沒做完，需求必須 **>L**，矛盾。所以模型內 U≤1 充分；加上前段必要性，即為充要條件。這個證明靠的是每個區間的需求上界，並非「平均沒滿就一定沒事」。

前例 U=15/16，故 EDF 可保證所有 jobs；實機切換成本、鎖阻塞、多核心或 D<T 都會改變上述前提，不能照搬結論。

相對期限短於週期時，低利用率也可能無解

令 τ_1、τ_2 都是 C=2、T=10，但 D=3，同步在 0 釋放。U=0.4，然而兩筆 job 都須在 t=3 前完成，合計需要 4 ms；單 CPU 在 [0,3) 只供應 3 ms。**不論 RM、EDF 或其他政策，至少有一筆超時**。

C=2、T=10、D=3：兩筆都在 0 到達，3 之前需要 4



供給 3 < 需求 4；t=3 時 $J_{2,0}$ 還剩 1

對一般 $D_i \le T_i$，長度 L 內可能「釋放且到期」的最大需求為

$$\mathrm{dbf}_i(L) = \max\left(0, \left\lfloor \frac{L - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

首筆需留 D_i 時間；若 $L \ge D_i$，此後每隔 T_i 可再放一筆，故得到上式。此例 L=3，每個 task 的 dbf=2，合計 4>3，已足以證明不可排程。只檢查某一個 L 通過不能證明全部區間都通過。

即時分析的第一步是把**需要多少 CPU、何時到達、最晚何時交付**分開寫。priority inheritance 可處理特定鎖造成的優先權反轉，但不會把這個 4>3 的容量矛盾消掉。

6.7　系統名稱，不能代替選取規則

本節固定以下歷史版本與事件模型：Solaris TS/IA、Windows 7，以及 Linux 3.2 CFS。比較的是特定模型的行為，並非現代版本的效能排名。

系統案例	選取結構與主要規則	不可省略的條件
Solaris TS/IA	類別優先權轉全域優先權；用盡片額與睡醒可改等級	等級表、片額、睡醒規則
Windows 7	32 級 ready queue；同級輪流、動態提升	基礎等級、boost、tick 單位
Linux 3.2 CFS	依 vruntime 的樹與公平選取提示	權重、計時器與喚醒門檻

原書不把上述三者化成一個共同 RR 片額。以下 CFS 案例先把時序條件固定，再逐事件核對。

Linux 3.2 CFS：先固定輸入，再推切換時刻

單 CPU；P1/P2 在0就緒，需求8/4 ms；P3需求2 ms，睡眠至9。切換、interrupt 與 timer jitter 成本皆0。

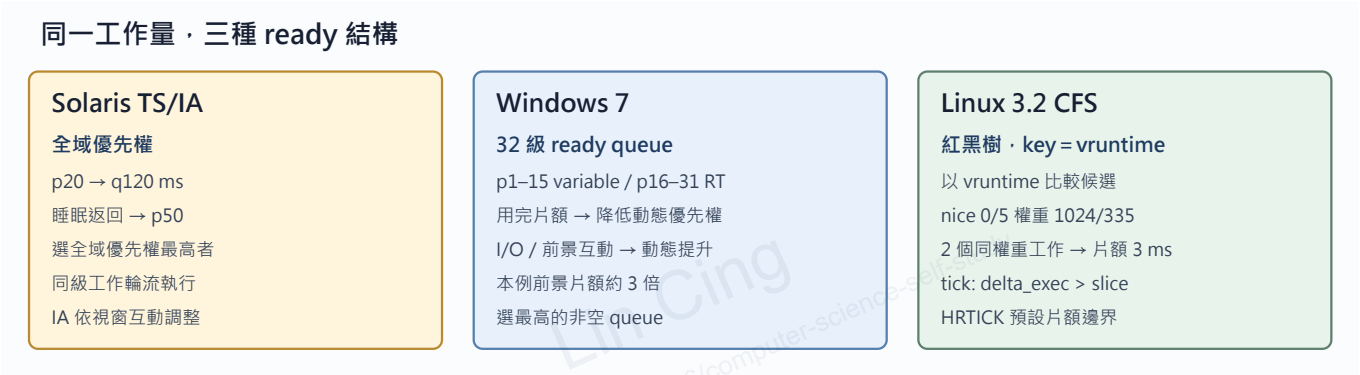
輸入	本例設定
選取鍵與平手	三者初始 vruntime=0，同鍵時P1先入樹
權重	nice=0，各為1024
計時器	HRTICK啟用；wakeup granularity=1 ms
睡眠規則	GENTLE_FAIR_SLEEPERS=true

令n為runnable數，固定target latency=6 ms、minimum granularity=0.75 ms：

$$period = \begin{cases} 6\ \text{ms}, & n \le 8, \\ 0.75n\ \text{ms}, & n > 8, \end{cases} \qquad ideal_i = period \frac{weight_i}{\sum_j weight_j}.$$

n=2時各得3 ms，n=3時各得2 ms。這是可執行片額，並非全系統固定quantum。所有鍵同移一個常數不改變相對次序；真實新建工作的START_DEBIT另受建立規則影響。

三種 ready 結構，不能共用一套片額規則



本例 current 不放在 ready tree；仍可執行的舊current，才會依更新後vruntime回樹。睡眠者要等喚醒，完成者直接離開。選定current後，ready欄不再包含它。

CFS 逐事件表：每次切換都要有原因

列事件處理及選取之後的狀態。ready不含current；current/片額中的片額單位為ms。

t／事件	current／片額	其他 ready	min_vruntime	選取理由
0 初始	P1／3	P2@0	0	同鍵，P1先入樹
3 HRTICK	P2／3	P1@3	0	P1用滿片額
6 HRTICK	P1／3	P2@3	3	同鍵，既有最左者
9 喚醒／HRTICK	P3／2	P2@3、P1@6	3	P3放回0，領先差6>1
11 P3完成	P2／3	P1@6	3	3<6
12 P2完成	P1／6	空	6	只剩P1，需求2
14 P1完成	無／—	空	8	保留最後基準

t=9 的喚醒：鍵值小於基準，基準卻不倒退

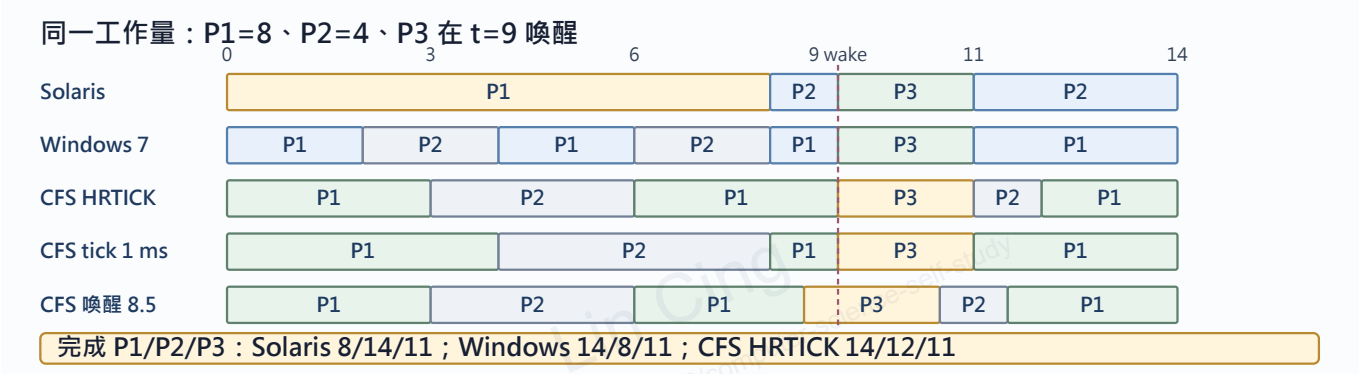
P1自6執行至9，vruntime由3增至6；P2仍ready，鍵為3。P3睡眠前的鍵為0，放回時套用

$$v_{P3} = \max(v_{sleep}, min_vruntime - 3) = \max(0, 3 - 3) = 0.$$

階段	current	其他可選者	min_vruntime
喚醒前	P1@6	P2@3；P3尚在睡眠	3
比較鍵值	P1@6	P2@3、P3@0	3
選取後	P3@0	P2@3、P1@6	3

6−0>1 ms，所以P3在9搶占；它在8仍睡眠，不能提早執行。min_vruntime是單調前進的基準，不是任何瞬間所有鍵值的最小值，故不隨P3回到0。完整時間線在下一頁，完成為14/12/11。

把不同計時器的時間線並排比較



6.7　tick、HRTICK 與 sleep placement 的敏感度

同一個 workload 在不同 timer 模式下不能假裝只有一張答案。

設定	Gantt	completion P1/P2/P3	機制差異
HRTICK，P3 wake t=9	P1[0,3)、P2[3,6)、P1[6,9)、P3[9,11)、P2[11,12)、P1[12,14)	14/12/11	queued HRTICK 在 slice 邊界直接重排
無 HRTICK，1 ms tick，P3 wake t=9	P1[0,4)、P2[4,8)、P1[8,9)、P3[9,11)、P1[11,14)	14/8/11	delta_exec=4>3 才週期 tick 搶占
HRTICK，P3 wake t=8.5	P1[0,3)、P2[3,6)、P1[6,8.5)、P3[8.5,10.5)、P2[10.5,11.5)、P1[11.5,14)	14/11.5/10.5	wakeup lead 5.5>1 ms，真正發生喚醒搶占

check_preempt_tick 的 delta_exec > ideal_runtime 是嚴格不等號。沒有 HRTICK 時，1 ms tick 在 t=3 只看到 3>3 為 false，故 P1 到 t=4 才被一般 tick 搶占。HRTICK 由另一個高解析度 timer 在 slice 被排定的時刻觸發 queued 路徑，該路徑直接設定重排，不重新套用嚴格不等號。

週期檢查、喚醒比較與 buddy 提示

週期 tick 的第二個條件是：delta_exec≥minimum_granularity 後，若 current 的 vruntime 大於 leftmost 且差值超過 ideal_runtime，也可重排。這個條件處理錯過喚醒邊界的工作，不能取代喚醒比較。

last/next/skip buddy 是 cache locality 與避免過度排程的候選提示，不是另一個無視 vruntime 的 priority。此例在 P3 尚睡時 n<8，未形成一般 next buddy；真正 wakeup 超過 granularity 時才把 next buddy 指向 P3。若 buddy 與 leftmost 的 virtual-time 差距太大，仍不能越過公平界線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.7　nice、weight 與跨案例比較

Linux 3.2 的 nice 不直接加減 priority queue；它改變 weight。v3.2 的 nice=0 weight 為 1024，nice=5 weight 為 335。若只有這兩個 task runnable 且 period=6 ms：

task	nice	weight	ideal_runtime	實體每 1 ms 增加的 vruntime
A	0	1024	6×1024/1359≈4.521 ms	1 ms
B	5	335	6×335/1359≈1.479 ms	1024/335≈3.057 ms

所以 nice 低者取得約 75.35% 的 CPU，nice 高者取得約 24.65%；權重並非線性 CPU percentage。n>8 時 period 隨 runnable 數延長，讓 minimum granularity 避免 slice 無限變短。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一工作量，為何沒有唯一的系統時間表？

案例	固定版本規則	同一 logical workload 的 completion P1/P2/P3
Solaris TS/IA	p20/q120 ms；sleep 返回提升到 p50	8/14/11
Windows 7	32 級 queue；normalized 2/6 tick；I/O boost	14/8/11
Linux 3.2 CFS	HRTICK；nice=0；6 ms period；P3 t=9 wake	14/12/11

這張表比較的是事先明定的事件模型，不是三個產品的 benchmark。若 Linux 關閉 HRTICK，上述 completion 改為 14/8/11；若 P3 在 t=8.5 醒來，則為 14/11.5/10.5。可見「同一 workload」仍不足以推出唯一 Gantt，tick 模式與事件時刻必須先固定。

6.8　比較政策：用同一批輸入，也要看未完成者

方法	能回答什麼？	不能直接保證什麼？
確定性時間表	指定輸入下每次選擇與指標	其他到達分布也相同
佇列模型	在到達／服務假設下推導平均	未滿足假設時的尾端延遲
模擬	指定成本與分布下的政策行為	未建模的硬體或工作負載
實作量測	特定版本與機器上的觀測	所有機器與未來負載

固定三筆工作皆在0抵達、需求5/2/3 ms且無切換成本，SJF平均等待7/3、RR q=2為13/3；完成三筆皆需10 ms。平均等待較低，不能推成吞吐量也較高。若只統計已完成的短工作而漏掉長工作，平均值還會受到選樣偏差影響。

6.9　CPU 一直忙，等待仍可能變長

沿用 RR q=2、到達全為0、初序P1/P2/P3，只把三筆需求同乘k。每次取隊首，執行 min(2,剩餘需求)，未完成才放回隊尾。忽略切換成本，時間單位為 ms。

k；bursts	completion P1/P2/P3	waiting P1/P2/P3	throughput jobs/ms	本批p99首 CPUresponse	最大 waiting
1；5/2/3	10/4/9	5/2/6	3/10	4	6
2；10/4/6	20/10/16	10/6/10	3/20	4	10
3；15/6/9	30/16/25	15/10/16	1/10	4	16

三批的首次取得 CPU 都是0/2/4，利用率都100%；最大等待卻由6增至10再增至16。首次回應好，並不代表長工作很快完成。這是有限批次的比較，不能據此證明持續到達下也沒有 starvation。

把 k=2 的每個時間片列出來

完成為20/10/16，減去需求10/4/6，得到等待10/6/10。每筆只在 ready 或 running，故 W=T−CPU需求；若加入 I/O，必須再扣除 blocked 的時間。

時間	執行工作	執行後剩餘需求	結果
0–2	P1	8	回隊尾
2–4	P2	2	回隊尾
4–6	P3	4	回隊尾
6–8	P1	6	回隊尾
8–10	P2	0	完成
10–12	P3	2	回隊尾
12–14	P1	4	回隊尾
14–16	P3	0	完成
16–18；18–20	P1	2；0	最後兩片沒有跨工作切換

練習與參考資料

練習題與詳解分別見本章獨立 PDF。

- Operating System Concepts, 9th Edition，第 6 章，印刷頁 261-312。
- *Operating System Concepts*, 9th Edition, Section 6.7, pp. 290-299：Linux CFS、Windows 7 與 Solaris class/priority/quantum 的版本案例。
- Linux v3.2 `kernel/sched_fair.c`：`__sched_period` 與 `sched_slice`、`place_entity`、`check_preempt_tick`、HRTICK 與 wakeup preemption。核心行段見 495-543、952-1015、1119-1153、1862-1905 與 2520-2662。
- Linux v3.2 `kernel/sched.c`：nice −20 至 19 的 `prio_to_weight`；nice=0 為 1024、nice=5 為 335。
- Microsoft Learn: Priority Boosts 與 Scheduling Priorities：Windows priority/quantum 與 foreground/I/O boost。
- Oracle Solaris Time-Sharing Class：time-sharing priority/quantum 與 sleep/CPU-use 調整。

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study